

Context basics in JavaScript

Let's say we have some function `func` that uses `this` inside:

```
1 function func() {  
2   console.log(this.value);  
3 }
```

What does `this` point to in this function? And we don't know. And JavaScript doesn't know. And the function itself doesn't know. That is, at the moment the function is created, what exactly `this` points to is not defined. And it will be determined only when this function is invoked.

Let us have some kind of input:

```
1 <input id="elem" value="text">
```

Let's bind our function `func` to this input so that it executes when the input loses focus. Now, at the moment the function is executed, `this` will point to our input:

```
1 let elem = document.querySelector('#elem');  
2  
3 elem.addEventListener('blur', func);  
4 function func() {  
5   console.log(this.value); // will display 'text' on focus loss  
6 }
```

But after all, we can have not only single input, but several:

```
1 <input id="elem1" value="text1">  
2 <input id="elem2" value="text2">
```

And to each of these inputs we can bind our function `func`. In this case, for the first element, `this` at the time of the function call will point to first, and for the second - to second.

In practice, this means that `this` inside the function depends on which of the inputs we lost focus on:

```
1 let elem1 = document.querySelector('#elem1');  
2 elem1.addEventListener('blur', func);  
3  
4 let elem2 = document.querySelector('#elem2');  
5 elem2.addEventListener('blur', func);  
6  
7 function func() {  
8   console.log(this.value); // will output either 'text1' or 'text2'  
9 }
```

In general, this feature `this` is very convenient - we create just one function and bind it to any number of inputs. If `this` did not refer to the element in which the event occurred, then we would not have succeeded - we would have to create our own function with the same code for each input!

Unbound function context in JavaScript

Let's see what happens if you specify `this` in the function not bound to any element, like this:

```
1 function func() {  
2   console.log(this);  
3 }  
4  
5 func();
```

In this case, the result depends on whether we have a `strict` mode or not. If the mode is non-strict, then `this` will store a reference to `window`:

```

1 function func() {
2   console.log(this); // reference to the window
3 }
4
5 func();

```

And if the mode is strict, then `undefined` will be stored in `this`:

```

1 "use strict";
2
3 function func() {
4   console.log(this); // undefined
5 }
6
7 func();

```

From the above it would seem that in strict mode `this` will always be `undefined`. It's not the case! If you just output `this` outside the function, then it will have a reference to `window` regardless of the mode:

```

1 "use strict";
2 console.log(this); // there is reference to the window in this

```

Summary

You can write `this` in any function without causing any JavaScript errors. But what exactly will lie in `this` is not defined until the moment the function is called. Moreover, with different function calls, `this` can be different values. It all depends on the context in which the function was called.

Context loss in JavaScript

Let's say we have a function within a function. Let's call the outer function `parent` and the inner one `child`:

```

1 function parent() {
2   function child() {
3
4   }
5 }

```

From previous lessons you know that if you set any variable in the outer function, it will be available in the inner one

```

1 function parent() {
2   let str = 'abcde';
3
4   function child() {
5     console.log(str); // shows 'abcde'
6   }
7   child(); // calls the inner function
8 }
9
10 parent(); // calling the outer function

```

There is, however, a nuance: the inner function has access to all variables of the outer one, but does not have access to `this`. That is: if an external function is bound to some DOM element, then `this` in it will point to this element, but `this` of the inner function will not!

What would `this` of the inner function point to then? Answer: it will be equal to `undefined` (in strict mode) since the function is not bound to anything.

Let's check in practice. Let us have an input:

```

1 <input id="elem" value="text">

```

Let's attach the function `parent` to this input, which will be called when the input loses focus:

```
1 "use strict";
2
3 let elem = document.querySelector('#elem');
4 elem.addEventListener('blur', parent);
5
6 // Called on loss of focus:
7 function parent() {
8   // there will be some code
9
10  function child() {
11    // there will be some code
12  }
13  child(); // calls child function
14 }
```

Let's print the contents of `this` to the console in two places: inside the `parent` function and inside the `child` function:

```
1 "use strict";
2
3 let elem = document.querySelector('#elem');
4 elem.addEventListener('blur', parent);
5
6 function parent() {
7   console.log(this); // will display the reference to our input
8
9   function child() {
10     console.log(this); // will display undefined
11   }
12   child();
13 }
```

Run this code, lose focus to the input and look at the console - you will see that the first `console.log` will print a reference to our input to the console, and the second will just `undefined`. This situation, when `this`, in an unexpected way for us, indicates not what we need, is called *context loss*.

No 1

©jsPmCxCL

Let now in one and in the second function we will display `value` of the input. Determine what will be displayed in the lines of code marked with comments:

```
1 "use strict";
2
3 let elem = document.querySelector('#elem');
4 elem.addEventListener('blur', parent);
5
6 function parent() {
7   console.log(this.value); // what will output?
8
9   function child() {
10     console.log(this.value); // what will output?
11   }
12   child();
13 }
```

Solving the context problem in JavaScript

In the previous lesson, you saw how the context is lost in the inner function. Let's solve this problem. There are several solutions. Let's take a look at them.

Context with variable in JavaScript

The solution is as follows: in the outer function, write `this` to any variable and this variable will be available in the inner function, like all variables (usually this variable is called `self`). Thus, we will pass `this` from the outer function to the inner one:

```
1 "use strict";
2
3 let elem = document.querySelector('#elem');
4 elem.addEventListener('blur', parent);
5
6 function parent() {
7   console.log(this.value); // shows 'text'
8
9   let self = this; // write this to any variable, for example, to self
10
11   function child() {
12     console.log(self.value); // shows 'text'
13   }
14   child();
15 }
```

№ 1

©jsPmCxVS

Let this code be given:

```
1 <input id="elem" value="3">
2
3 "use strict";
4
5 let elem = document.querySelector('#elem');
6 elem.addEventListener('blur', func);
7
8 function func() {
9   alert( square() );
10
11   function square() {
12     return this.value * this.value;
13   }
14 }
```

The author of the code wanted the square of the number from input `value` to be displayed on the screen when focus was lost. However, for some reason, when focus is lost, an error is generated in the console. Please fix the code author's mistake. Write a text in which you will explain to the author of the code why this error occurred.

Context with passing parameter in JavaScript

There is another solution to the problem. Let's make the function `child` take a parameter:

```
1 function child(param) {
2   // here will be a code
3 }
```

And when calling this function, we will pass `this` into it:

```

1 function parent() {
2   child(this); // pass the this as parameter
3
4   function child(param) {
5     // the param variable contains the passed content of this
6   }
7 }

```

Since the call to `child` is carried out in the parent function, the passed `this` points out what we needed. Then the `this` gets into the `param` parameter and will be available inside the function in this form.

Here is the final code:

```

1 let elem = document.querySelector('#elem');
2 elem.addEventListener('blur', parent);
3
4 function parent() {
5   child(this); // passes the this as parameter
6
7   function child(param) {
8     console.log(param.value); // prints the value of the input
9   }
10 }

```

No 1

©jsPmCxPPS

Take the code from the previous task and fix the code problem using the second method you learned.

Context with arrow function in JavaScript

The third solution to the problem is to use [arrow functions](#), which appeared in JavaScript not so long ago. Such functions, among other things, do not have their own context, but retain the context of the parent. Let's fix our problem with an arrow function:

Arrow functions in JavaScript

Let's now look at *arrow functions*, which simplify the function syntax.

In the following code example, an ordinary function is written first, and the corresponding arrow function is written second (both functions do the same thing):

```

1 let func1 = function(num1, num2) {
2   let result = num1 * num2;
3   return result;
4 }
5
6 let func2 = (num1, num2) => {
7   let result = num1 * num2;
8   return result;
9 }

```

If the function has one line of code, then in arrow functions you can not write `return` and curly braces:

```

1 let func1 = function(num1, num2) {
2   return num1 * num2
3 }
4
5 let func2 = (num1, num2) => num1 * num2;

```

If there is only one parameter of the arrow function, parentheses can be omitted:

```

1 | let func1 = function(num) {
2 |   return num * num;
3 | }
4 |
5 | let func2 = num => num * num

```

If the function has no parameters at all, you need to write empty parentheses:

```

1 | let func1 = function() {
2 |   console.log('!!!');
3 | }
4 |
5 | let func2 = () => console.log('!!!')

```

Context with arrow function in JavaScript

The third solution to the problem is to use [arrow functions](#), which appeared in JavaScript not so long ago. Such functions, among other things, do not have their own context, but retain the context of the parent. Let's fix our problem with an arrow function:

```

1 | let elem = document.querySelector('#elem');
2 | elem.addEventListener('blur', parent);
3 |
4 | function parent() {
5 |   console.log(this.value); // shows 'text'
6 |
7 |   let child = () => {
8 |     console.log(this.value); // shows 'text'
9 |   }
10 |   child();
11 | }

```

No 1

©jsPmCxAFS

Take the code from the previous task and fix the code problem using the third method you learned.

Binding a context via the call method in JavaScript

So we've covered how `this` actually works. Let's now look at methods that allow you to force the context in which the function is called (that is, force it to say what `this` is equal to).

The first method we'll cover is called `call`. Let's see how it works with an example. Let us have an input:

```

1 | <input id="elem" value="text">

```

Let's get a reference to this input and write it to the variable `elem`:

```

1 | let elem = document.querySelector('#elem');

```

Let's now make a function `func`, inside which we will display `this.value` to the console:

```

1 | function func() {
2 |   console.log(this.value);
3 | }

```

So far, our function doesn't know what `this` refers to. Now, if we attached it through `addEventListener`, then yes. But we won't do that. Instead, we simply call our function, telling it that `this` should be equal to `elem`.

This is done like this: `func.call(elem)`. This code is equivalent to simply calling the function `func` like this: `func()`, only with the condition that `this` is equal to `elem`.

So, the syntax of the call method is: `function.call(object)`. Let's put it all together:


```

1 | let elem = document.querySelector('#elem');
2 |
3 | function func() {
4 |   console.log(this.value); // shows the input value
5 | }
6 | func.call(elem);

```

No 1

©jsPmCxCM

Given a function:

```

1 | function func() {
2 |   console.log(this.value);
3 | }

```

Given three inputs:

```

1 | <input id="elem1" value="text1">
2 | <input id="elem2" value="text2">
3 | <input id="elem3" value="text3">

```

Using the `call` method and the `func` function, display the `value` of each of the inputs.

Call method with parameters in JavaScript

Now let the function `func` take some parameters, let's call them `param1` and `param2`:

```

1 | function func(param1, param2) {
2 |   console.log(this.value + param1 + param2);
3 | }

```

When calling a function via `call`, you can pass these parameters like this:

```

1 | func.call(elem, param1, param2);

```

No 1

©jsPmCxCMP

Let the following code be given:

```

1 | <input id="elem" value="hello">

1 | let elem = document.querySelector('#elem');
2 |
3 | function func(surname, name) {
4 |   console.log(this.value + ', ' + name + ' ' + surname);
5 | }
6 |
7 | func(); // should output here 'hello, John Smit'

```

Add the `call` method to the last line so that 'hello, John Smit' is displayed. The word 'hello' must come from the input value, 'John' and 'Smit' must be function parameters.

Binding a context via the apply method in JavaScript

Let's now look at the `apply` method for context binding. It works in much the same way as the `call` method. The difference is that in `apply`, the parameters are passed as an array, rather than listed with a comma. Depending on the task, one or the other method is convenient.

Here is an example of passing parameters to the `apply` method:

```
1 | func.apply(elem, [param1, param2]);
```

No 1

©jsPmCxAM

Let the following code be given:

```
1 | <input id="elem" value="hello">

1 | let elem = document.querySelector('#elem');
2 |
3 | function func(surname, name) {
4 |   console.log(this.value + ', ' + surname + ' ' + name);
5 | }
6 |
7 | func(); // should output here 'hello, John Smit'
```

Add the `apply` method to the last line so that 'hello, John Smit' is displayed. The word 'hello' must come from the value input, 'John' and 'Smit' must be function parameters.

Binding a context via the bind method in JavaScript

The following `bind` method allows you to permanently bind a context to a function. As a result, this method returns a new function, inside which `this` will have a hard-coded value.

Let's look at an example.

Let us have an input:

```
1 | <input id="elem" value="text">
```

Let the reference to this input be written into the variable `elem`:

```
1 | let elem = document.querySelector('#elem');
```

Suppose we also have the following function `func`:

```
1 | function func(param1, param2) {
2 |   console.log(this.value + param1 + param2);
3 | }
```

Let's make a new function using `bind`, which will be a copy of the function `func`, but `this` in it will always be equal to `elem`:

```
1 | let newFunc = func.bind(elem);
```

Now the variable `newFunc` contains a function. Let's call it by passing '1' in the first parameter, and '2' in the second (remember that `elem` contains an input with a value equal to 'text'):

```
1 | newFunc('1', '2');
```

Let's get it all together:

```
1 | let elem = document.getElementById('elem');
2 |
3 | function func(param1, param2) {
4 |   console.log(this.value + param1 + param2);
5 | }
6 |
7 | let newFunc = func.bind(elem);
8 | newFunc('1', '2'); // shows 'text12'
```


It is not necessary to write the result of `bind` into a new function `newFunc`, you can simply overwrite `func`. After that, `func` will be the same function as it was, but with a hard-coded `this`:

```
1 | func = func.bind(elem);
```

№ 1

©jsPmCxBM

Let the following code be given:

```
1 | <input id="elem" value="hello">

1 | let elem = document.getElementById('elem');
2 |
3 | function func(name, surname) {
4 |     console.log(this.value + ', ' + name + ' ' + surname);
5 | }
6 |
7 | // Let's write a construct with bind() here
8 |
9 | func('John', 'Smit'); // should output here 'hello, John Smit'
10 | func('Eric', 'Luis'); // should output here 'hello, Eric Luis'
```

Write a construction with the method `bind` in the indicated place so that `this` inside the `func` function always points to the input from the `elem` variable

Getting of attributes in JavaScript

Learn the theory at the following links:

[getAttribute](#)